

Architectural Scaling & Optimization

SYS-304 Milestone 3 | 20 September 2026

Benchmark Results

- **Naive:** original pickle API, uncached (Frozen phase 2 model).
- **Optimized:** Native XGBoost archive with fused preprocessing and TTL/LRU caching. Dynamic batching is disabled by default.

Experimental Model (Unused)

An experimental model (optimized_xgb_pipeline.pkl) was developed using domain-specific feature engineering, a $\log_{1p}(\text{SalePrice})$ target, and a scikit-learn pipeline, and is retained solely as a benchmark comparison point. It is omitted from production deployment and API serving due to higher preprocessing overhead and slower inference times. Legacy notebooks and training files are separate from the active serving and Markdown report workflows.

Methodology

Each model artifact was loaded in a fresh process and tested with the same raw input records on the same machine. The benchmark used 100 repeated prediction calls per trial across three trials with single-threaded XGBoost. The pickle implementations constructed a DataFrame during prediction, while the native implementation used saved preprocessing metadata and sparse CSR input. Timing results report median latency. Memory was measured separately during an untimed prediction pass using 30 samples, and artifact size was measured from the saved files.

The native archive produced the best time and runtime-memory results. Its one-row median was 0.53 ms compared with 22.13 ms for Phase 2, while its 64-row median was 4.50 ms compared with 23.82 ms. Throughput increased from 2,674.63 to 13,298.01 rows/s. Native loaded RSS was 276.1 MB with 278.8 MB active sampled RSS, compared with 284.4 MB and 372.0 MB for Phase 2. The historical pickle was smaller on disk, but it was slower and had higher active memory, so file size alone is not a useful measure of serving efficiency.

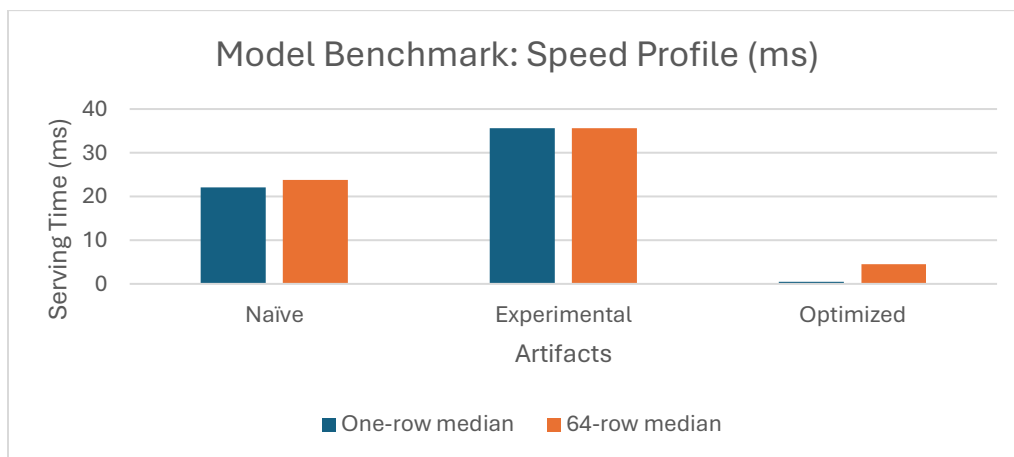


Figure 1: Artifacts Speed Profiles

Artifact	One-row median (ms)	64-row median (ms)	64-row throughput (rows/s)
Naive	22.13 ms	23.82 ms	2,674.63 rows/s
Experimental	35.59 ms	35.65 ms	1,710.98 rows/s
Optimized	0.53 ms	4.50 ms	13,298.01 rows/s

Artifact	Loaded RSS	Active RSS	On-disk size
Naive	284.4 MB	372.0 MB	102.9 KB
Experimental	282.4 MB	373.6 MB	69.3 KB
Optimized	276.1 MB	278.8 MB	95.8 KB

Figure 3: Artifacts Memory Profile Table

The API benchmark launched a fresh local Uvicorn process for each trial and used persistent HTTP connections with the same request generator. It tested unique requests and a repeated ten-record workload at concurrency 1 and 32, using 160 requests per trial across three trials. Latency percentiles, successful throughput, process-tree RSS, errors, cache hits, and batching counters were recorded. Only successful requests were included in the latency and throughput calculations, and all recorded requests succeeded. The load generator and server ran on the same machine.

The native cache configuration improved throughput over the uncached Phase 2 service at both concurrency levels. At concurrency 32, cache-only throughput reached 100.99 requests/s for unique requests and 113.60 requests/s for repeated requests, compared with 44.54 and 43.26 requests/s for Phase 2. Batching was inconsistent, it helped some unique-request runs but reduced repeated-request throughput, and the isolated ablation reduced throughput in both workloads while increasing tail latency. Cache-only therefore remains the measured default. A two-worker deployment was not tested and would use separate model instances, caches, and queues.

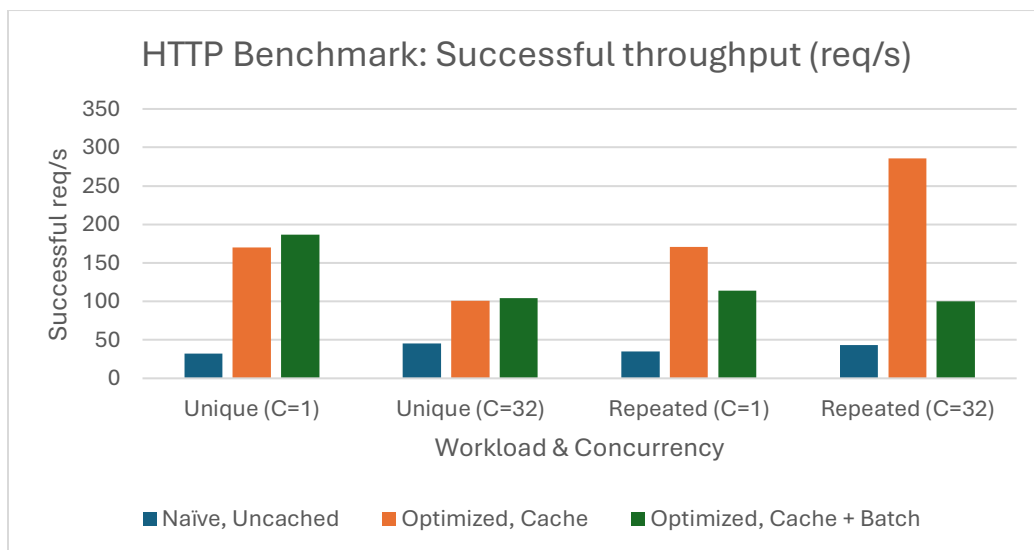


Figure 4: Successful local throughput for different configurations

Workload	Configuration	Successful req/s
Unique	Native, neither	105.83
Unique	Native, batching only	81.05
Unique	Native, cache only	100.99
Repeated	Native, neither	103.63
Repeated	Native, batching only	93.46
Repeated	Native, cache only	113.60

Figure 5: Successful throughput comparison (Non vs Cache vs Batching).

Deployment and limitations

The native artifact reproduces the naive pipeline and schema. It changes inference format and the serving preprocessing path, not the trained model or accuracy. The backend image uses `backend/requirements-runtime.txt`, selecting CPU-only XGBoost on amd64 Linux and regular XGBoost elsewhere. Docker Compose defaults to one Uvicorn worker and one inference worker. Cache state is process-local, the optional batch queue is created only when batching is enabled. Additional Uvicorn workers would have independent model instances, cache and queue state, so increasing web concurrency trades memory and cache sharing for process-level concurrency. Docker image build and container execution are unverified, compose configuration and application behavior were checked independently. The reported RSS comes from local Python processes, not from a trimmed image. These local single-thread measurements are comparisons, not production capacity guarantees.

Architecture

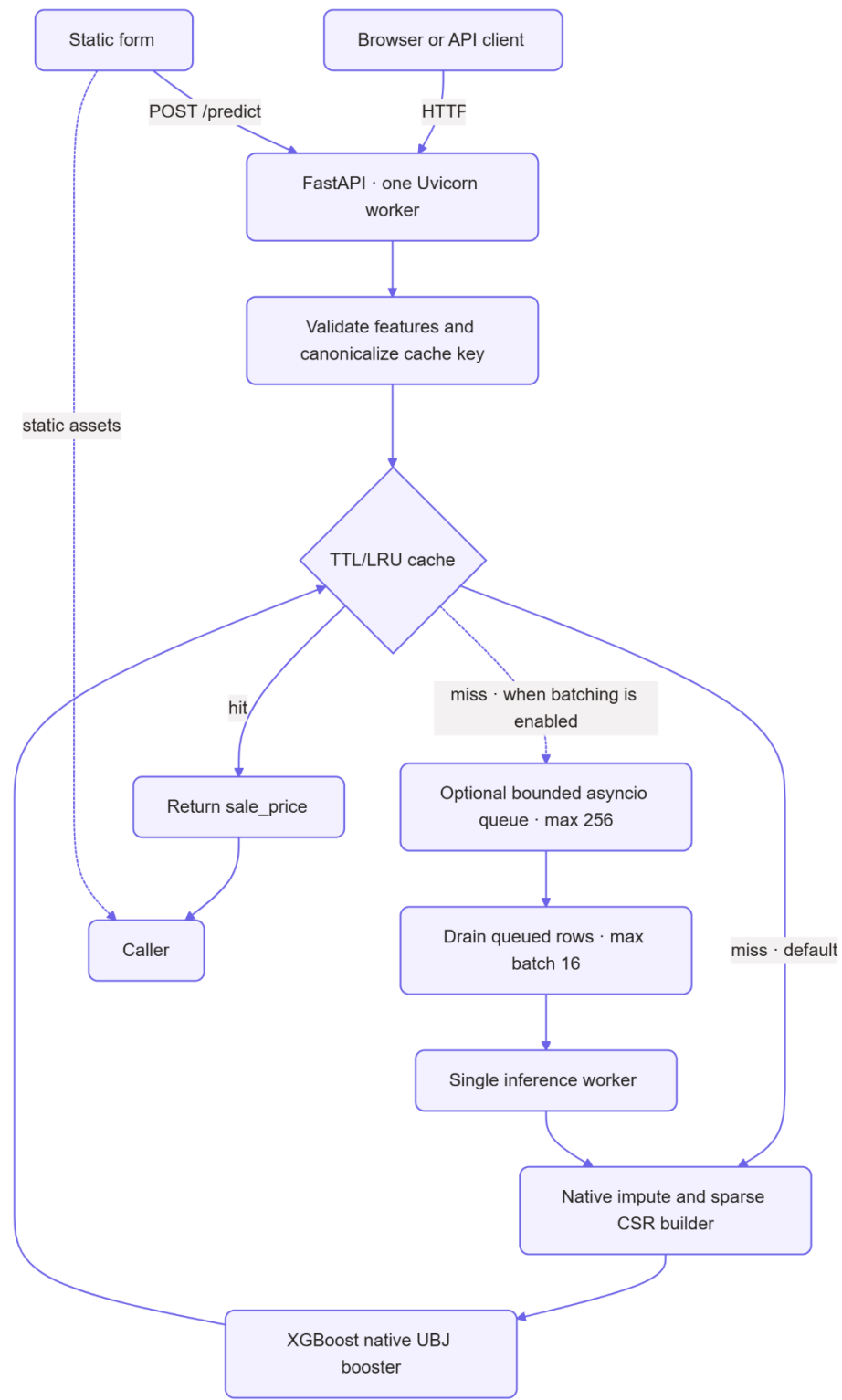


Figure 6: Top-down flow of the serving design.

The exact-input cache is process-local, and concurrent identical misses share one in-flight prediction. On the default path, a cache miss runs directly through native inference. When dynamic batching is enabled, misses instead enter the bounded queue and are grouped before inference. The measured workload did not show a reliable batching gain, so the default keeps the setting disabled. Compose runs one API worker so cache state and, when enabled, queue state is shared.

Two Uvicorn workers are a possible deployment setting for more process-level concurrency on a multi-core host. They do not share the process-local cache, in-flight map, or batch queue, and each worker loads its own model and runtime. That can improve concurrency when one process is saturated, but it also increases memory use and can reduce cache sharing and batch fill. This report did not include a two-worker run, so the web concurrency at 1 worker remains the measured default, and a two-worker deployment should be benchmarked against the expected traffic mix before adoption.